

Hallucination Detection in Large Language Models

April 2026

Word count: 4967

Hallucination Detection in Large Language Models

I Introduction

Today, billions of people around the world use AI chatbots as tools for tasks ranging from creative writing to information inquiry. ChatGPT, released in 2022, started the frantic chase of generative AI. In the next year or so, large tech companies around the world released hundreds of AI models, many of which were capable of chatting fluently with the user. Till this moment, each of these AI models consists of billions of parameters that are trained and fine-tuned.

It is not difficult to see that AI has increasingly provided significant convenience. Researchers from different areas see the great opportunities that generative AIs present. The potential use of AI in healthcare, education, and many more domains has both excited and worried the researchers (Yan et al.; Zhui et al.; Maity and Saikia; Geng et al.).

With the opportunities come the challenges that can lead to great costs. Not only researchers, but most users of AI models have also realized that they do not always provide the correct information. Frankly, they make mistakes and talk as if they have full confidence in what they are saying, confusing or convincing the uninformed user. In some situations, this is less of a problem, as the incorrectness of some information is insignificant. However, in other situations, it could cause tremendous consequences. An article from The New York Times reviewed a scandal of a lawyer who uses AI in the court briefs containing fabricated precedents and fictional legal citations (Dewan). This irresponsibility led to severe consequences. Thus, it is essential for researchers to find ways to prevent such things from happening.

Common approaches to this problem include the detection of non-factual information and the mitigation of these hallucinations. Our study chooses to focus on the detection of hallucination. In this work, we propose to use deep learning models to analyze

outputs from the AI chatbots, also known as Large Language Models (LLMs), to identify the factuality of the contents of the text. The deep learning models we will use consist of an MLP model and a hybrid model that combines MLP layers and an attention layer. For inputs, we implement both consistency detection and uncertainty detection on the outputs of the LLM. Since the LLM outputs rely completely on the final layer token probabilities, we further propose to compare the two sampled responses by using attention matrices derived from the log probabilities in the hybrid models. The results of the models are discussed.

II Related Works

Hallucination in Large Language Models is conceptually defined as the fabricated and nonfactual information in LLM outputs, also defined as the inconsistency between the output of the ground truth function and the output of an LLM (Cossio 5; Xu et al. 3). Such hallucination was considered inevitable. Well-grounded algorithms could only mitigate, not eliminate, hallucination. Studies around this topic categorize LLM hallucinations into two main types: Extrinsic and Intrinsic hallucinations (Cossio 9). Although there are other categories for hallucination classification (Factuality & Faithfulness hallucination), this study primarily focuses on these two.

Extrinsic hallucination refers to situations when LLMs make factual mistakes, creating nonexistent information or failing extrinsically (Cossio 9). This kind of hallucination is most popularly dealt with by Retrieval Augmented Generation (RAG), retrieving external information to help LLMs make accurate responses, or to examine the factuality of their response (Ji et al.; Lavrinovics et al.). Knowledge graphs have been a common RAG technique. Ji et al. used knowledge graphs as a means to integrate and organize data, which is then used to provide

information for the LLM to answer correctly. However, it poses problems as the dataset requires constant updating and checking. Dependencies on such datasets would introduce the risk of retrieving the wrong knowledge, causing the model to be prompted with information that is completely erroneous.

Intrinsic hallucinations refer to instances where the LLM's answer contradicts the information given by the user or when the generated answer does not fully align with the expectations of the user (Cosio 9). This can be dealt with self-check methods, where models are prompted to check for themselves if the generated answers include hallucinations. These methods can also be used to detect extrinsic hallucinations if different models are used for detecting, as they used different databases during their training process (Friel and Sanyal).

Many studies also prefer to use **black-box methods** when detecting or mitigating hallucinations. Black-box methods use the logits¹ of LLMs to understand the properties of their answer. Farquhar et al., Senior Research Scientists at DeepMind, propose a method that groups the final text choices into groups with similar semantics. The model inflates the significance of the difference between the meaning of sentences instead of the difference between exact tokens². This is more efficient as the LLM will not need to choose from texts that have the same meanings, and it will focus more on identifying hallucinations from different semantic groups.

Consistency detection focuses on comparison among multiple samples. In Manakul et al's paper, the authors, Ph.D. in Information Engineering from the University of Cambridge, propose the approach SelfCheckGPT, which illustrates this method. They compare the discrepancies between the answers generated from the same prompt. Several techniques are used to convert these discrepancies into metrics,

enlarging them so that it is possible for hallucination evaluation. The ideology behind this is that LLMs tend to hallucinate when answering the same question differently. Therefore, detecting if the model is answering consistently or how much the model varies every time would show how much it trusts its output. Building on Manakul's work, Ledger and Mancinni use Monte Carlo simulation, which also tests LLMs' internal confidence about their generation. The paper suggests recording the variance and the precision through the logits of the multiple answers it generates. Sriramanan et al. propose several metrics that analyze the hidden states, the attention maps, and the perplexity of the model, giving a score that determines how much it believes in its answers (Sriramanan et al.). These consistency detection methods use logits to calculate an indicator of the correlation between outputs, potentially giving an accurate and promising score on the model's confidence. However, these methods can lead to high computational costs. SelfCheckGPT, for example, it takes more than five generations to determine the factuality of one result, making it resource wasteful and time-consuming.

Uncertainty Analysis concentrates on analyzing the logits in an attempt to understand the uncertainty of the model's answers. Kuhn et al. introduce the idea of semantic entropy, which puts answer choices from the LLM with similar meanings into groups. Later on, researchers from Nature uses these techniques in hallucination detection. Their methods cluster answers with similar semantics in order to prevent confusion when the classification model determines its class (Farquhar et al.; Kuhn et al.). Furthermore, Sriramanan et al. also use traditional uncertainty measures for the output logits. They designed a logit entropy formula that is able to calculate lexical entropy and combined it with measures

¹ Logits are the raw outputs of the LLM. They are not processed through a sigmoid function which means it can be any real number. Sigmoid functions convert these raw outputs into probabilities.

² Token is the smallest unit of meaningful text. E.g. pre-fix. It can be smaller than a word.

that analyzed the internal attention maps of LLMs. These methods do seem less computationally costly, yet they are less effective and may include bias due to inference based only on lexicality.

Most advanced strategies for hallucinations use trained deep learning models for detection, many of which use white box³ techniques that aim to analyze the internal states of LLMs; other techniques focus on evaluating the difference between answers from the same prompt, but they both concentrate on assessing how confident the model is with their generated answers. Each and every one of these methods is flawed in some way; combining them to gain a comprehensive view of the analyzed text is a novel and promising approach. In this study, we will try to integrate these evaluation metrics and solve the problem for both intrinsic and extrinsic hallucinations.

III Methodology

A. Target LLM Choice

To gather data for the training of a hallucination detection model, we choose Llama-3.2-3B-Instruct-Turbo, a target model that is lightweight and optimized for fast conversations. With 3 billion parameters, it is less complex and more hallucinogenic. The model is accessed through the Together API, which provides output logprobs⁴ of the generated answers. Other choices like GPT-2 from OpenAI are considered too. However, although GPT-2 allows access to its internal structure, its number of parameters is far less than that of the currently popular models. Thus, choosing a light Llama model would be more up-to-date and efficient compared to other large models.

B. Data Collection

We use queries from TruthfulQA as prompts that is given to the target model to induce hallucination. The dataset contains queries of many

categories, including common misconceptions, and many specialized areas of knowledge, such as Politics, Economics, Sociology. This allows for the dataset collected to be more comprehensive, as it increases the variety of the hallucination examples the dataset is composed of.

The answers from the target model are collected with the log probabilities. They are then analyzed by a human labeler who determines if it is hallucinogenic. The standard of the labels is based on the comprehensive taxonomy of hallucinations given by Cossio (Cossio 15). A total of 816 answers were gathered and used.

The dataset is divided into a training set, a validation set, and a testing set. The training set is used to train the classification model, the validation set is used to make sure that the model is not overfitting and schedules the learning rate⁵, and the testing set tests the performance of the final trained model. Separating the dataset helps to prevent data leakage that could make the testing and validation of models unreliable.

C. Detection Metrics

We propose to analyze the output of the LLM by dissecting it into two parts: the output logits and the generated text itself. The output logits are analyzed through logit entropy, while the output tokens are used for consistency measures. We integrate these two approaches so the combined score would better reflect the factuality of the model’s answers. Logit entropy, the equation proposed by Sriramanan et al. as mentioned above, is a measure to determine uncertainty (7).

$$\text{LogitEnt}(x, k) = -\frac{1}{m-n+1} \sum_{i=n}^m \sum_{j=1}^k p_f(x_i^j | x_{<i}) \log p_f(x_i^j | x_{<i})$$

This equation is a simple way to calculate the total entropy of the model’s generated output by

³ Generally, they are methods that expose and analyze the inner working mechanisms of subjects. White box methods for hallucination detection uses the inner structures and attention maps of the LLM.

⁴ Logprobs are short for log probabilities. They are calculated by taking the log of token probabilities. Here log is the logarithm of base e. $\text{logprobs} = \log(\text{token probabilities})$

⁵ The variable that controls the rate the model is learning from the gradient. A scheduler changes the learning rate according to the loss of the validation epoch after running a training epoch.

considering only the top-k tokens. In simple terms, the formula calculates how surprised the LLM is with the output tokens. $p_f(x_i^j | x_{<i})$ is the token

probability⁶, while $\log p_f(x_i^j | x_{<i})$ is the logprob.

Both of the x are the jth choice of the ith token in the generated text, given the context of previous tokens. The logprobs calculate the confidence of the model with the token choice. The more unconfident the model is, the more likely the logprobs approach negative infinity. The probabilities give weight to the logprobs. The token that is assigned the largest probability would have the most weight, where even a slight uncertainty would cause the final score to change significantly. They are summed for all tokens in the text and averaged.

The logit entropy score is then combined with the cosine similarity between the two answers generated from the same prompt to create a dataset. The text embeddings⁷ for cosine similarity are calculated using sentence-BERT models (Reimers and Gurevych). Text embeddings can be interpreted as a set of parameters that represent the semantic meaning of the short text. Cosine similarity can then be measured by the following formula:

$$S_c(x, y) = \frac{x \cdot y}{||x|| \times ||y||}$$

The numerator is calculated using a dot product of vectors x and y. In the denominator, $||x||$ and $||y||$ calculate the magnitude of the vectors, while $||x|| \times ||y||$ is the regular product of these two values. Cosine similarity computes the alignment of the angle between two vectors and outputs a number between -1 and 1. 1 represents completely aligned meanings of the x and y, vice versa. This metric is used to measure how well the two generated texts agree with each other.

The logit entropy measures the model's perplexity with one output, while the cosine similarity

measures model confidence through two outputs. To some extent, the two metrics complement each other.

D. Model Architecture

To diversify the experiment, we chose two model structures for the same input texts. Both of the models work on token probabilities and cosine similarities; one is more complicated. Overfitting and underfitting are two common pitfalls for model training.

Overfitting refers to when a model does very well on the training data but fails the validation set. It happens when the model learns all the noise in the training data and loses the ability to generalize to the validation data. **Underfitting**, on the other hand, refers to occasions when the training fails, and the model is unable to generate correct outputs as expected. Underfitting happens when the model is either not complicated enough for the data, or the data contains no useable info for an expected output.

The first model architecture we chose is a simple multi-layer perceptron (MLP) network. The network consists of three MLP layers, a total of one input layer, two hidden layers, and one output layer. The input layer consists of two features: logit entropy and cosine similarity. The two hidden layers and the output layer are connected using the ReLU activation function, introducing nonlinearity. See illustration in Appendix I.A.

$$ReLU(x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$

The output is analyzed using a sigmoid function that outputs a value between 1 and 0, which can be converted to a percentage of how likely the model is hallucinating.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

To stabilize training, we use batch normalization after the hidden layers. These layers prevent the distribution of inputs of each layer from changing dramatically, speeding the convergence of training.

The second model architecture we chose to use is

⁶ Token probabilities are scores assigned by the LLM to every token in its dictionary for choosing the best succeeding token.

⁷ Text embeddings are vectors(matrices) that contains meanings of the text, often generated by models.

a hybrid neural network. The hybrid neural network consists of an attention layer (Vaswani et al.) and three MLP layers. The attention layer directly compares the two generated logits by using a scaled dot product.

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

Where the dot product of Q and the transposed matrix of K is scaled by $\sqrt{d_k}$. Here, Q stands for the query matrix, and K stands for the key matrix⁸. The softmax function⁹ is used to transform the vectors from the resulting matrix to proportions. This will then be multiplied by a value matrix using the dot product. See illustration in Appendix I.B.

Let the matrices constructed by the two responses from the LLM that contain the top 5 token probabilities for each generated token be response 1 and response 2, respectively. Then, $Q = V = \text{Response 1}$, and $K = \text{Response 2}$. The attention layer mainly compares the two responses and outputs a meaningful attention matrix. The matrix is pooled into a single value, then combined with the cosine similarity and the logit entropy score, which act as features of the input layer of the MLP network. The MLP network has the same architecture as the first model, except that it takes three features as its input layer.

Due to underfitting and overfitting issues, the model structures can be remodified in the experiment. If overfitting exists, we consider adding dropout layers¹⁰ to MLP layers for both models and reducing model complexity by reducing the number of layers. If underfitting exists, we consider increasing the complexity of the models by adding a convolutional layer¹¹ that pools the attention layer output in

the hybrid model.

E. Training

The optimizer used during training is Adam, while the loss function used is the Binary Cross-Entropy Loss function (BCELoss)¹². BCELoss function is commonly used for binary classification tasks, and Cross-Entropy Loss is used for multi-class classification tasks. For the reason that we are classifying LLM answers into two categories, hallucination or non-hallucination, we will use the BCELoss.

F. Evaluation Benchmarks

One common evaluation benchmark for binary classification tasks is the Area Under the Receiver Operating Characteristic Curve (ROC or AUROC) score (Farquhar et al. 627; Mir 12). The ROC curve graphs the True Positive rate against the False Positive Rate. The area under the ROC curve ranges between 0 and 1. A score of one indicates perfect performance of the model, correctly classifying every input. A score of 0.5 indicates the model acts no better than a random selector.

Another benchmark commonly used for binary class classification tasks is the F1 score, which combines precision and recall (Mir 12). Precision is the proportion of correctly identified hallucinations within all the predicted hallucinations (Mir 12).

$$Precision = \frac{TP}{TP + FP}^{13}$$

Recall is the proportion of correctly identified hallucinations within the total of real positives. Here, since we are constructing a model that detects hallucination, recall is more important than precision (Mir 12).

⁸ K^T is the transposed matrix of K.

⁹ The softmax function converts matrices of numbers into proportions which sums up to 1. This function is also used in the final layer of LLMs to determine token probabilities.

¹⁰ Dropout layer randomly drops neurons during training for each batch. It is commonly used to prevent overfitting

¹¹ Convolutional Neural Networks (CNN) are used widely in image processing. They are extremely good at analyzing matrices. In the original design, a normal pooling layer is used to pool the attention output where there are no parameters and gradients. CNN layers do have parameters. Although this study uses this layer later on in the experiments, we will not explain explicitly its mechanism.

¹² An optimizer uses back propagation calculate gradients which will change the parameters of the model. Learning rates control the rate in which the gradients change the parameters.

¹³ True Positives (TP), False Positives (FP)

$$Recall = \frac{TP}{TP + FN}^{14}$$

The F1 score is the harmonic mean of Recall and Precision (Mir 12).

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

We use both the AUROC score and the F1 score to evaluate the model’s performance. The AUROC score and F1 are calculated based on the testing dataset to prevent data leakage, as mentioned.

IV Experiments & Analysis

After running multiple training tests with different configurations, it was found that almost all the models performed poorly. Learning rates tried are 0.1, 0.05, 0.01, 0.005, 0.001; model dimensions tried are 32, 64, 128, 256; thresholds that are used to distinguish model output distributions were set to 0.5.

For these initial tries, most hybrid models choose to output 1 (tagging the text hallucinogenic), no matter what the input is, for this maximizes their total accuracy by guessing only the majority category. Out of the 60 trained models that have different model configurations, more than half of them have an accuracy of exactly 0.66, a precision of 0.66, and a recall of 1. This means that most models choose to ignore the input and output only 1 since the created dataset has a proportion of 2:1 for the data categories. Figure 1 is the distribution of a hybrid model. This distribution is a great example as the model

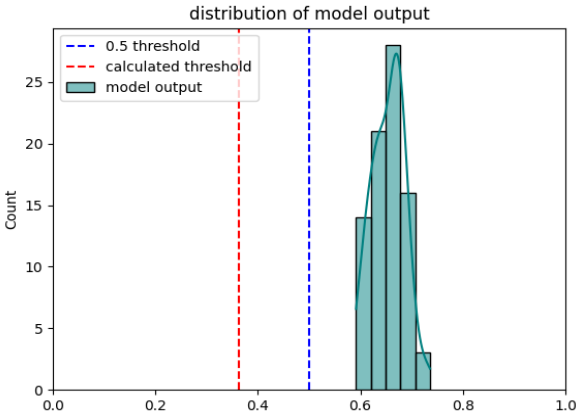


Figure 1: The distribution of a hybrid model with dimension of 128, trained with learning rate of 0.005.

chooses to output all values larger than the 0.5 threshold and has an approximate normal curve rather than an expected bimodal distribution. MLP models do not do better than the hybrid models. They also put all outputs above the threshold to maximize accuracy and F1-scores. Their output distributions are similar to those of the hybrid model in Figure 1.

However, there are exceptions. Figure 2 shows an example of a hybrid model that outputs a gapped distribution where the model tries to output similar to the expected value. The distribution is also reasonably left-skewed because of the 1:2 proportion in the dataset. Regardless of the clusters in the distribution, the F1-score, recall, and precision are relatively unsuccessful.

The AUROC score and the ROC curves of one of the MLP and hybrid models are shown in Figure 3. They have almost the same curve score. The hybrid model with an AUROC of 0.65 hardly surpasses the MLP model with a score of 0.64, and both models barely exceed a random model at 0.5.

A possible reason for the model’s bad performance is that the model is not complex enough to analyze the log-probability matrices; hence, we increase the hybrid model’s complexity by adding a Convolutional layer after the attention layer to pool the output matrix, as mentioned, see illustration in Appendix I.C. The Convolutional channels tried are

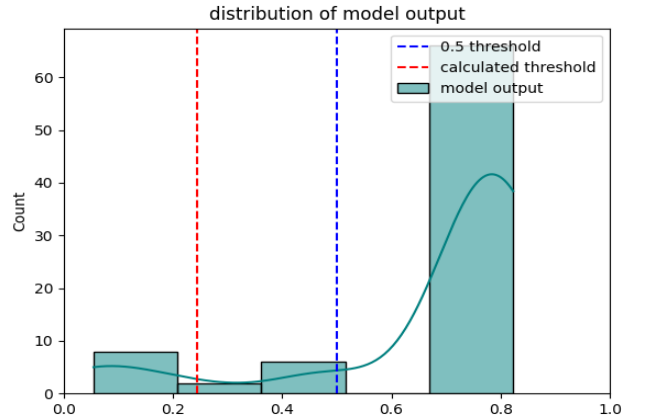


Figure 2: The distribution of a hybrid model with dimension of 32, trained with learning rate of 0.1.

¹⁴ False Negatives (FN)

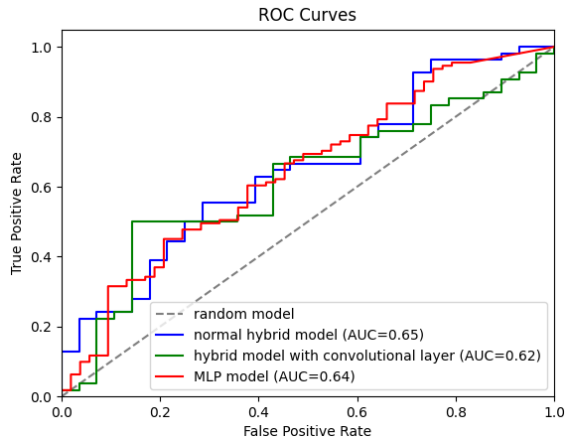


Figure 3: the ROC curves and the AUROC scores for the hybrid model and MLP model in figure 1 & 2 and a hybrid model with convolutional layer. All without weights.

32, 64, together with the other learning rates and model dimensions.

The models did not do better after adding a convolutional layer to the hybrid model. Figure 3 also shows the difference between the ROC curves of a model that implements the convolutional layer and a model that does not implement the convolutional layer. The distribution of the output of the model with an added convolutional layer is normal shaped. The convolutional layer does not help the model to perform better; instead, it lowers the model's AUROC score. The hybrid models with the convolutional layer do the same as the original hybrid models and MLP models by choosing to output only positive.

To try to improve the models and stop them from being biased because of the uneven distribution of the dataset, we implemented weights on the BCELoss function. Thus, for each incorrect decision the models make on the non-hallucinogenic answers will receive double the punishment for incorrect decisions made on hallucinogenic answers. All model dimensions are tried again, including those with the convolutional layers.

Figure 4 is the result for the models when a weight is added to the loss function. Again, the models performed poorly, underfitted the data, and failed to improve significantly. Among the three models, the MLP model performed the best, staying above

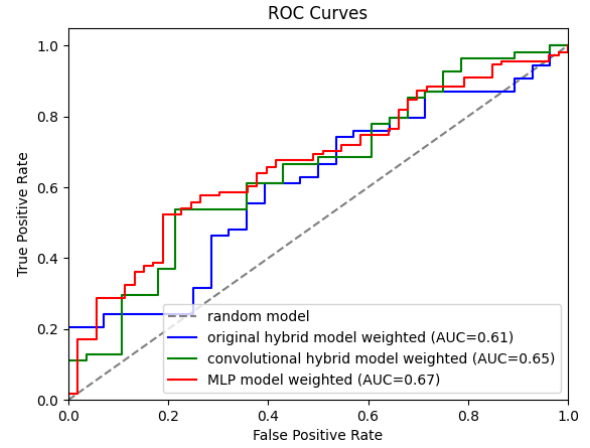


Figure 4: the ROC curves and the AUROC scores for an original hybrid model a MLP model and a hybrid model with convolutional layer. All with weights.

the ROC curves of the two other models most of the time. The curves of the unweighted convolutional hybrid model in Figure 3 and the weighted MLP model in Figure 4 rise above the random model baseline at a greater magnitude at lower false positive rates, indicating that they are able to find real positives with a threshold that minimizes false positives. As a result, they are less likely to cause false alarms compared to the other models shown.

Although most models did not excel in the AUROC and F1 scores, their output distribution is closer to what we want. Figure 5 shows the distribution of a hybrid model without the convolutional layer. The distribution is clearly bimodal with the 0.5 threshold separating the two clusters. This indicates that the model tries to learn patterns and tries to distinguish the non-hallucinated answers from the hallucinated answers. However, the model does poorly on the

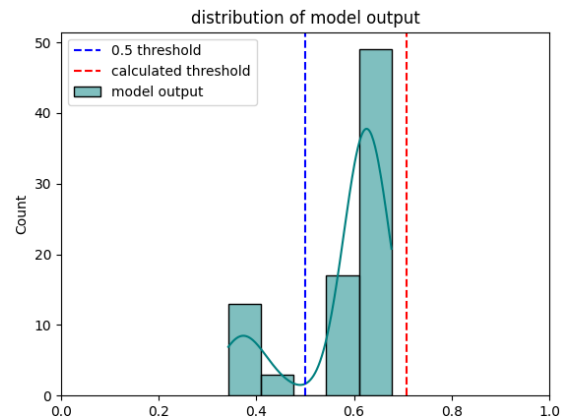


Figure 5: the model distribution of one of the original hybrid models with weight. The model has a dimension of 64 and learning rate of 0.001.

scores, scoring 0.77 and 0.61 on the F1 score and the AUROC score, respectively. Nonetheless, adding weights stopped the models from biasing towards one class.

Furthermore, we also try to change the threshold that distinguishes the continuous distribution of the model into two discrete categories output so the F1 score is maximized rather than using the default threshold of 0.5. The new threshold is calculated based on running the validation dataset, which searches through all the possible F1 scores and gets the best one. Because of the small validation dataset, the calculated threshold may vary greatly and could cause worse performance than the 0.5 threshold. The red dashed lines in Figures 1, 2, and 5 are the calculated thresholds. It is evident that some of them are worse than the 0.5 threshold.

Two fundamental problems are possible for underfitting. It can be because of either the model’s architecture and dimensions, or the calculated indices, and the attention matrix of logprobs does not contain information for the model to distinguish the answer’s factuality. In other words, the data is full of noise. For the first reason, the dimensions we tried are already quite large, so it is more likely to be a problem with the model architecture. The second reason is more probable if changing the model architecture does not work.

V Limitations

There are several limitations that could cause the model’s bad performances. Problems in the self-created dataset exist. These problems include the inaccuracies of the human labels of hallucination and the size of the dataset. Most of the time, it is difficult to identify the factuality of the answers from the LLMs. Labeling the non-hallucinated text as a hallucination or the hallucinated text as non-hallucination would both cause the model to be confused when analyzing the input features. Moreover, the small dataset of 816 data points is too small for such a

difficult task, making it even harder to optimize the model.

Other problems with the selected detection metrics exist as well. Cosine similarity can be biased since it is only related to the lexical similarity, not the semantic similarity. Logit entropy also focuses on lexical meanings and can be biased. Furthermore, using the logprobs directly can be a clumsy way that introduces too much unnecessary noise when given to the models.

VI Conclusion

This study tries to implement two neural networks to analyze the output logprobs from the large language models. The two models were designed to identify if the text outputs from the models are hallucinations. The first model is a three-layer MLP model that analyzes the logit entropy and the cosine similarity of two sampled outputs from the same answer. The second model is a hybrid model that combines an attention layer with an MLP layer. Later on, to test how the models perform based on their complexity, we added a convolutional layer that replaces the pooling layer between the attention layer and the MLP layer for the hybrid model. The input for the hybrid models is similar to the MLP model since both models take logit entropy and cosine similarity as inputs, but it is also designed to receive the full matrices of logprobs for the two sample outputs from the LLM.

The dataset is self-created based on the TruthfulQA, with a total of 816 queries. These questions act as hallucination-inducing prompts, which were then given to the target LLM. Logits are collected and analyzed. A human labeler is tasked to separate the hallucinated texts and the non-hallucinated texts and label them.

The two models performed poorly and suffered greatly from underfitting. They struggled to distinguish between positives and negatives. The MLP models and the hybrid models failed to achieve high

AUROC and F1 scores. Before adding weights, both models tend to bias their outputs towards tagging the inputs as hallucinogenic. This is caused by the imbalance in the dataset, as two-thirds of the data are marked as hallucination positive. Thus, by implementing weights on the BCELoss function, we improved the output distributions. Nevertheless, the models did not perform significantly better.

The limitations of this study are evident. The dataset consists of only 816 data points, which is significantly less than what is needed to train such complex models. Moreover, the labels that are assigned to each output are likely to be flawed. Although we do use a rubric for evaluating the answers from the LLM, it is impossible to avoid making mistakes. These incorrect labels may be a reason for the bad performance of the two models.

With these limitations, we advise **future researchers** to focus on the following points:

- Use a larger dataset for queries. Thus, the model can be trained on a more sophisticated dataset.
- Examine and label the answers from the LLM carefully and cross-check them with more than two human labelers. Ensure that most of the tags are labeled according to the ground truth. This helps reduce the likelihood of introducing false labels, which would lead to confusion in the models.
- Use a better model structure to analyze the logit outputs, and use more model dimensions to increase complexity. Instead of taking logprobs directly as input, consider analyzing them with more metrics.
- Focus the detection in one area specifically to decrease the difficulty of the task. For example, logical errors or self-contradicting claims. Moreover, researchers can choose to analyze intrinsic or extrinsic hallucinations specifically.
- Use more than one output for consistency detection. In methods like SelfCheckGPT, the authors choose to use more than 5 answers as inputs. This

would help the model better analyze the LLM's confidence.

In conclusion, this study's implementation of the novel approach that aims to detect hallucinations of LLMs by combining logprob attention matrices, cosine similarity, and logit entropy fails. However, future researchers can reference our model architectures, our novel ideas, and avoid the mistakes we have made during our research process.

Works Cited

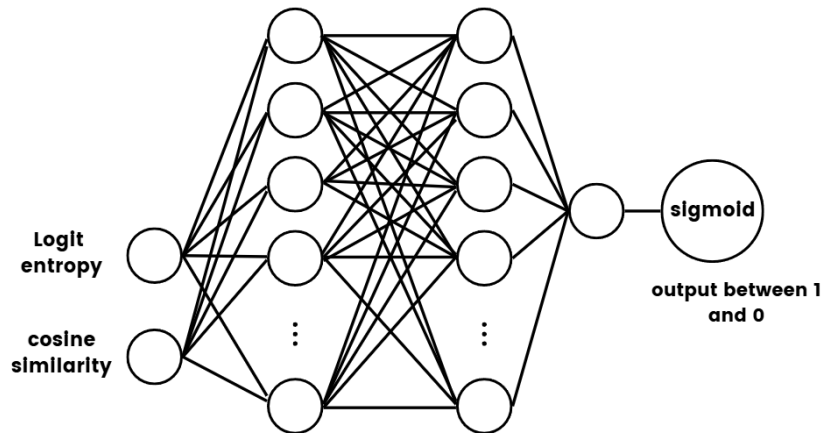
- Cossio, Manuel. "A Comprehensive Taxonomy of Hallucinations in Large Language Models." *arXiv:2508.01781*, arXiv, 3 Aug. 2025. *arXiv.org*, <https://doi.org/10.48550/arXiv.2508.01781>.
- Dewan, Shaila. "Prosecutor Used Flawed A.I. To Keep a Man in Jail, His Lawyers Say." *The New York Times*, 25 Nov. 2025, www.nytimes.com/2025/11/25/us/prosecutor-artificial-intelligence-errors-lawyers-california.html.
- Farquhar, Sebastian, et al. "Detecting Hallucinations in Large Language Models Using Semantic Entropy." *Nature*, vol. 630, no. 8017, June 2024, pp. 625–30. *Crossref*, <https://doi.org/10.1038/s41586-024-07421-0>.
- Friel, Robert, and Atindriyo Sanyal. "Chainpoll: A

- High Efficacy Method for LLM Hallucination Detection.” arXiv:2310.18344, arXiv, 22 Oct. 2023. *arXiv.org*, <https://doi.org/10.48550/arXiv.2310.18344>.
- Geng, Mingmeng, et al. “The Impact of Large Language Models in Academia: From Writing to Speaking.” arXiv:2409.13686, arXiv, 22 Oct. 2024. *arXiv.org*, <https://doi.org/10.48550/arXiv.2409.13686>.
- Ji, Ziwei, et al. *RHO (ρ): Reducing Hallucination in Open-Domain Dialogues with Knowledge Grounding*.
- Kuhn, Lorenz, et al. *SEMANTIC UNCERTAINTY: LINGUISTIC INVARIANCES FOR UNCERTAINTY ESTIMATION IN NATURAL LANGUAGE GENERATION*. 2023.
- Lavrinovics, Ernests, et al. “Knowledge Graphs, Large Language Models, and Hallucinations: An NLP Perspective.” *Journal of Web Semantics*, vol. 85, May 2025, p. 100844. *Crossref*, <https://doi.org/10.1016/j.web-sem.2024.100844>.
- Ledger, Grant, and Rafael Mancinni. “Detecting LLM Hallucinations Using Monte Carlo Simulations on Token Probabilities.” Institute of Electrical and Electronics Engineers (IEEE), 12 June 2024. *Preprints*, <https://doi.org/10.36227/techrxiv.171822396.61518693/v1>.
- Maity, Subhankar, and Manob Jyoti Saikia. “Large Language Models in Healthcare and Medical Applications: A Review.” *Bioengineering*, vol. 12, no. 6, June 2025, p. 631. *DOI.org (Cross-ref)*, <https://doi.org/10.3390/bioengineering12060631>.
- Manakul, Potsawee, et al. “SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models.” arXiv:2303.08896, arXiv, 11 Oct. 2023. *arXiv.org*, <https://doi.org/10.48550/arXiv.2303.08896>.
- Mir, Amir Hameed. “The Geometry of Truth: Layer-Wise Semantic Dynamics for Hallucination Detection in Large Language Models.” arXiv:2510.04933, arXiv, 6 Oct. 2025. *arXiv.org*, <https://doi.org/10.48550/arXiv.2510.04933>.
- Reimers, Nils, and Iryna Gurevych. “Sentence-BERT:

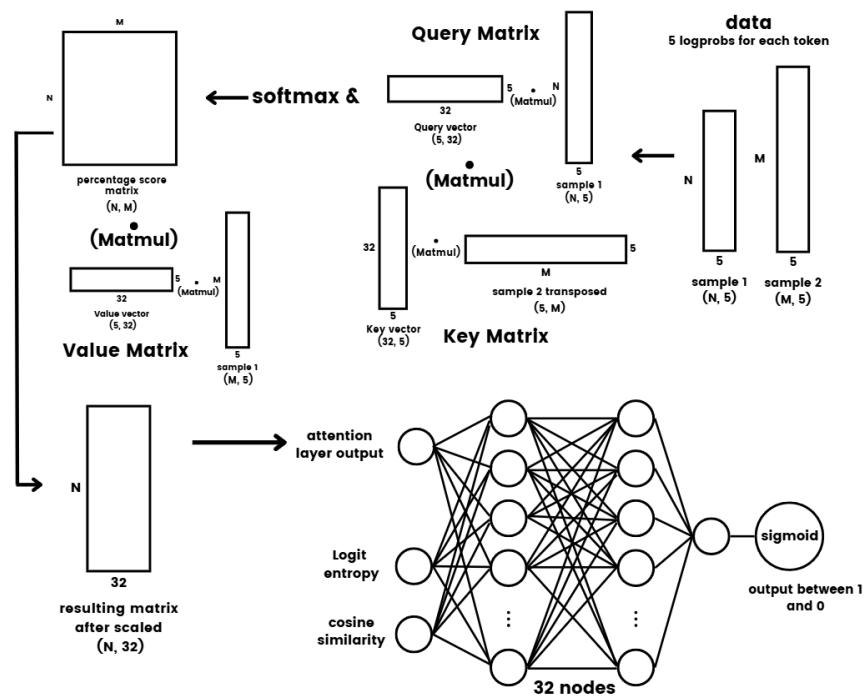
- Sentence Embeddings Using Siamese BERT-Networks.” arXiv:1908.10084, arXiv, 27 Aug. 2019. *arXiv.org*, <https://doi.org/10.48550/arXiv.1908.10084>.
- Sriramanan, Gaurang, et al. *LLM-Check: Investigating Detection of Hallucinations in Large Language Models*.
- Vaswani, Ashish, et al. “Attention Is All You Need.” arXiv:1706.03762, arXiv, 2 Aug. 2023. *arXiv.org*, <https://doi.org/10.48550/arXiv.1706.03762>.
- Xu, Ziwei, et al. “Hallucination Is Inevitable: An Innate Limitation of Large Language Models.” arXiv:2401.11817, arXiv, 13 Feb. 2025. *arXiv.org*, <https://doi.org/10.48550/arXiv.2401.11817>.
- Yan, Lixiang, et al. “Practical and Ethical Challenges of Large Language Models in Education: A Systematic Scoping Review.” *British Journal of Educational Technology*, vol. 55, no. 1, Jan. 2024, pp. 90–112. *arXiv.org*, <https://doi.org/10.1111/bjet.13370>.
- Zhui, Li, et al. “Impact of Large Language Models on Medical Education and Teaching Adaptations.” *JMIR Medical Informatics*, vol. 12, July 2024, pp. e55933–e55933. *DOI.org (Crossref)*, <https://doi.org/10.2196/55933>.

Appendix I

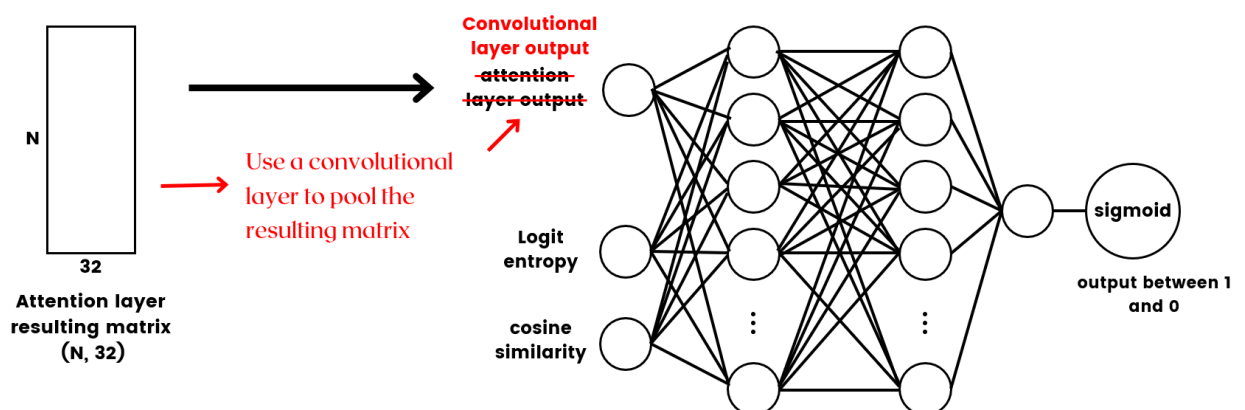
A. MLP model



B. Hybrid model without convolutional layer



C. Hybrid model with convolutional layer



Appendix II

Code and Datasets at Github:

[Artemis2467/ProjectCode: Novel Hallucination detection method \(github.com\)](#)

[Artemis2467/ProjectCode at new-features \(github.com\)](#)